Lab 5 Report

Chen Liangkun 23020036007. Lab group: 4.

Lab date: 20th December 2025. Report date: 22nd March 2026.

1 Lab Introduction and Team Information

This experiment explores the fundamental theories of computer vision and the performance of various state-of-the-art feature tracking techniques. It consists of two core parts: theoretical derivation of perspective projection and practical implementation of feature matching pipelines. In the theoretical section, the projection of a sphere onto an image plane is derived, demonstrating that while a sphere centered on the optical axis projects as a circle, it generally forms an ellipse due to perspective contraction. Additionally, the concept of vanishing points is analyzed, establishing that parallel 3D lines intersect at $(d_x/d_z, d_y/d_z)$ in the image plane unless they are parallel to the image plane itself.

In the practical section, the performance of several feature detection and matching algorithms—including SIFT, SURF, ORB, and FAST+BRIEF—is evaluated alongside the Lucas-Kanade (LK) optical flow tracker. Experimental results across multiple datasets reveal that FAST provides the highest keypoint density, while ORB offers an optimal balance between computational efficiency and matching robustness, consistently achieving high inlier ratios. The LK tracker further demonstrates the effectiveness of temporal consistency in maintaining robust feature sets through re-detection mechanisms. This experiment validates the integration of geometric projection theory with modern computer vision algorithms for efficient information extraction from image streams.

2 Individual Work

2.1 Practice with Perspective Projection

Consider a sphere with radius r centered at $P_c = [0, 0, d]^T$ with respect to the camera coordinate frame. The camera parameters are given as: principal point $(0, 0)$, focal length $f = 1$, pixel sizes $s_x = s_y = 1$, and zero skew $s_\theta = 0$. It is assumed that $d > r + 1$.

- **Task 1: Derive the equation describing the projection of the sphere onto the image plane.Solution:** Given the camera intrinsic parameters, the perspective projection equations can be simplified as:

$$u = f\frac{p_x^c}{p_z^c} = \frac{p_x^c}{p_z^c}, \quad v = f\frac{p_y^c}{p_z^c} = \frac{p_y^c}{p_z^c} \tag{1}$$

The surface of the sphere in the camera coordinate frame is defined by the equation:

$$(p_x^c - 0)^2 + (p_y^c - 0)^2 + (p_z^c - d)^2 = r^2 \tag{2}$$

Substituting $p_x^c = u \cdot p_z^c$ and $p_y^c = v \cdot p_z^c$ from Equation 1 into Equation 2, we obtain a quadratic equation in terms of p_z^c:

$$(u^2 + v^2 + 1)(p_z^c)^2 - 2d \cdot p_z^c + (d^2 - r^2) = 0 \tag{3}$$

The projection boundary (the silhouette) corresponds to the case where the projection ray from the optical center is tangent to the sphere. Mathematically, this implies that the quadratic equation must have exactly one real solution for p_z^c, meaning the discriminant must be zero:

$$\Delta = (-2d)^2 - 4(u^2 + v^2 + 1)(d^2 - r^2) = 0 \tag{4}$$

Solving for u and v:

$$4d^2 - 4(u^2 + v^2 + 1)(d^2 - r^2) = 0 \implies u^2 + v^2 = \frac{r^2}{d^2 - r^2} \tag{5}$$

The resulting equation describes a circle centered at the image origin with radius $R = \frac{r}{\sqrt{d^2 - r^2}}$.

- **Task 2: Discuss the projection when the sphere center is at an arbitrary location.**

  Solution: When the sphere center deviates from the optical axis, the symmetry of the projection is broken. From a geometric perspective, the set of rays tangent to the sphere forms a right circular cone with its vertex at the optical center. The image plane then intersects this cone at an oblique angle.

  As illustrated in the geometry of conic sections, an oblique section of a circular cone results in an ellipse. The general equation for the projection of a sphere centered at (x_0, y_0, z_0) can be derived as:

$$(x_0^2 - K)u^2 + (y_0^2 - K)v^2 + 2x_0y_0uv + 2x_0z_0u + 2y_0z_0v + (z_0^2 - K) = 0 \tag{6}$$

  where $K = x_0^2 + y_0^2 + z_0^2 - r^2$. This quadratic form represents an ellipse in the (u, v) image plane.

2.2 Vanishing Points

Consider two 3D lines that are parallel in the world frame. Under perspective projection, these lines typically intersect at a specific pixel in the image plane known as the vanishing point.

- **Task 1: Derive the generic expression of the vanishing point.Solution:** A 3D line can be parameterized as:

$$p(\lambda) = p_0 + \lambda \mathbf{d} \tag{7}$$

where $p_0 = [x_0, y_0, z_0]^T$ is a point on the line, $\mathbf{d} = [d_x, d_y, d_z]^T$ is the direction vector, and λ is a scalar parameter. The projection of any point on this line onto the image plane is:

$$u(\lambda) = \frac{x_0 + \lambda d_x}{z_0 + \lambda d_z}, \quad v(\lambda) = \frac{y_0 + \lambda d_y}{z_0 + \lambda d_z} \tag{8}$$

The vanishing point (u_{vp}, v_{vp}) is defined as the limit of the projection as the distance λ approaches infinity:

$$u_{vp} = \lim_{\lambda \to \infty} \frac{\frac{x_0}{\lambda} + d_x}{\frac{z_0}{\lambda} + d_z} = \frac{d_x}{d_z}, \quad v_{vp} = \lim_{\lambda \to \infty} \frac{\frac{y_0}{\lambda} + d_y}{\frac{z_0}{\lambda} + d_z} = \frac{d_y}{d_z} \tag{9}$$

Since the result $(u_{vp}, v_{vp}) = (\frac{d_x}{d_z}, \frac{d_y}{d_z})$ depends only on the direction vector $\mathbf{d}$ and is independent of the starting point p_0, all lines parallel to $\mathbf{d}$ will converge to the same vanishing point.

- **Task 2: Determine the condition under which parallel 3D lines remain parallel in the image plane.**

  Solution: 3D lines remain parallel in the image plane if and only if their vanishing point is at infinity. From the expressions derived above, this occurs when the denominator d_z equals zero:

$$d_z = 0 \tag{10}$$

  Physically, this condition means the 3D lines are parallel to the image plane. In this case, the lines do not converge to a finite pixel, and their projections remain parallel.

3 Team Work

In this section, we present the implementation of a robust feature tracking and matching pipeline designed to estimate camera motion. The system evaluates various feature detection and description algorithms, including SIFT, SURF, ORB, and FAST+BRIEF, as well as a differential optical flow tracker based on the Lucas-Kanade method. The implementation is modularized within the `FeatureTracker` base class located in the `lab5/feature_tracking/src` directory.

3.1 Feature Detection via SIFT

Feature detection is the process of extracting distinctive keypoints from an image. For reliable frame-to-frame tracking, a "good feature" must satisfy several criteria: **Repeatability** (invariant to geometric and photometric changes), **Saliency** (distinctive descriptors), **Compactness**, and **Locality** (robustness to occlusion).

Among various detectors, the Scale Invariant Feature Transform (SIFT) is particularly effective due to its invariance to image rotation and scale. In our implementation, the SIFT detector is initialized via `cv::SIFT::create()` and executed through the `detectKeypoints` member function.

3.1.1 Keypoint Refinement and Localization

SIFT identifies potential keypoints by searching for local extrema in the Difference-of-Gaussian (DoG) scale space. To ensure the stability of these points, the following refinement steps are applied:

1. **Sub-pixel Localization**: The location of each candidate keypoint is refined by fitting a 3D quadratic function to the local sample points using a Taylor series expansion:

$$D(\mathbf{x}) = D + \frac{\partial D^T}{\partial \mathbf{x}}\mathbf{x} + \frac{1}{2}\mathbf{x}^T\frac{\partial^2 D}{\partial \mathbf{x}^2}\mathbf{x} \tag{11}$$

where is the offset from the candidate point. 2. **Low-contrast Suppression**: Points with a response value below a certain threshold (e.g., 0.03) are rejected as they are sensitive to noise. 3. **Edge Response Removal**: To eliminate unstable edge points, SIFT examines the principal curvatures of the local Hessian matrix . A point is kept only if the ratio of curvatures is below a threshold :

$$\frac{\mathrm{Tr}(\mathbf{H})^2}{\mathrm{Det}(\mathbf{H})} < \frac{(r+1)^2}{r} \tag{12}$$

3.1.2 Orientation Assignment

To achieve rotation invariance, each keypoint is assigned a principal orientation. This is determined by calculating the gradient magnitude and orientation within a neighborhood:

$$m(x,y) = \sqrt{(L(x+1,y) - L(x-1,y))^2 + (L(x,y+1) - L(x,y-1))^2} \tag{13}$$

$$\theta(x,y) = \tan^{-1}\left(\frac{L(x,y+1) - L(x,y-1)}{L(x+1,y) - L(x-1,y)}\right) \tag{14}$$

The orientation corresponding to the highest peak in the local gradient histogram (and any peak within 80

The detection and description implementation in `sift_feature_tracker.cpp` is shown below:

Listing 1: SIFT Keypoint Detection and Description

```
void SiftFeatureTracker::detectKeypoints(const cv::Mat& img, std::vector<
    KeyPoint>* keypoints) const {
CHECK_NOTNULL(keypoints);
detector->detect(img, *keypoints);
}

void SiftFeatureTracker::describeKeypoints(const cv::Mat& img, std::vector<
    KeyPoint>* keypoints, cv::Mat* descriptors) const {
CHECK_NOTNULL(keypoints);
CHECK_NOTNULL(descriptors);
detector->compute(img, *keypoints, *descriptors);
}
```

The results of SIFT keypoint detection on two consecutive frames are visualized in Figure 1.

(a) Keypoints in Frame 1

(b) Keypoints in Frame 2

Figure 1: SIFT keypoint detection results for consecutive image frames.

3.2 Descriptor-based Feature Matching

Once descriptors are computed, we establish correspondences between frames. Given the high dimensionality of SIFT descriptors (128D), we utilize the Fast Library for Approximate Nearest Neighbors (FLANN) for efficient matching.

To filter out ambiguous or incorrect matches, we adopt the ratio test proposed by Lowe. For each descriptor in the first image, we find its two nearest neighbors (and) in the second image. A match is considered "good" only if the distance ratio satisfies:

$$\frac{\text{dist}(d_1)}{\text{dist}(d_2)} < \tau \tag{15}$$

where the threshold is typically set to 0.8. This ensures that the chosen match is significantly better than the next best alternative, effectively discarding features from repetitive textures or noise.

The implementation of the FLANN-based -nearest neighbor matching is as follows:

Listing 2: FLANN Matching with Lowe's Ratio Test

```
// Perform knnMatch with k=2
flann_matcher.knnMatch(descriptors_1, descriptors_2, *matches, 2);

// Filter matches using the 0.8 ratio threshold
for (auto& match : *matches) {
if (match.size() >= 2) {
if (match[0].distance < 0.8 * match[1].distance) {
good_matches->push_back(match[0]);
}
} else if (match.size() == 1) {
good_matches->push_back(match[0]);
}
}
```

The final matching results, visualizing the correspondences between the two frames, are shown in Figure 2.

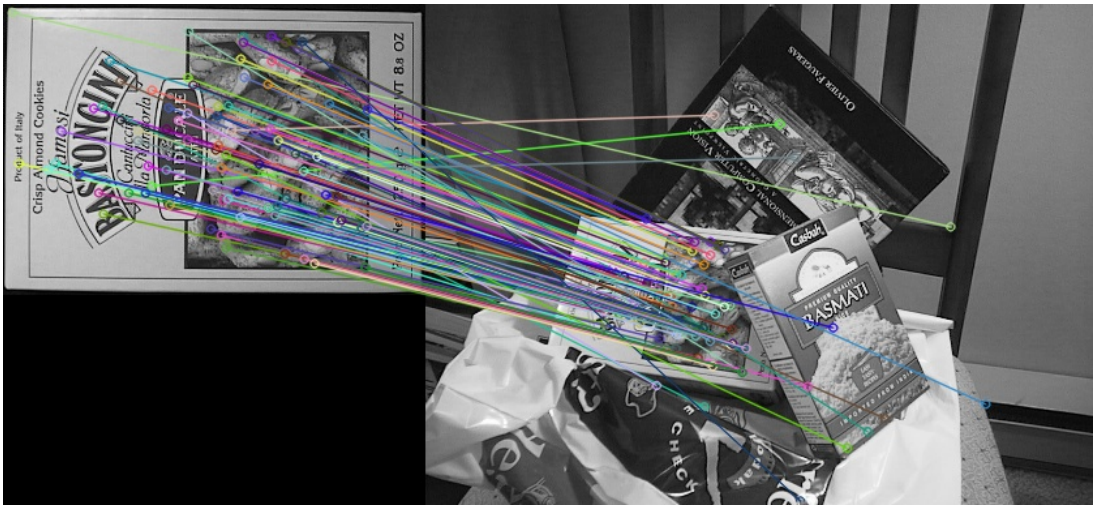


Figure 2: Result of SIFT feature matching using FLANN and Lowe's ratio test.

3.3 Keypoint Matching Quality Assessment

Following the initial feature matching, the set of correspondences often contains outliers—incorrect matches resulting from noise, repetitive textures, or significant viewpoint changes. To ensure the reliability of subsequent motion estimation, it is imperative to distinguish between *inliers* (correct correspondences) and *outliers* (false matches).

3.3.1 Outlier Rejection Logic

In this implementation, we utilize an inlier mask computation, which typically leverages robust estimation techniques such as RANSAC (Random Sample Consensus). By enforcing a geometric constraint (e.g., the Essential Matrix or Homography), matches that do not conform to the global camera motion are identified and masked.

The implementation details for extracting matched coordinates and computing the inlier mask are shown in the following snippet:

Listing 3: Extraction of matched keypoints and inlier mask computation

```
std::vector[cv::KeyPoint](https://www.google.com/search?q=cv::KeyPoint)
    matched_kp1, matched_kp2;
for (const auto& match : good_matches) {
matched_kp1.push_back(keypoints_1[match.queryIdx]);
matched_kp2.push_back(keypoints_2[match.trainIdx]);
}

std::vector<uchar> inlier_mask;
// Robust estimation function to compute the inlier mask
inlierMaskComputation(matched_kp1, matched_kp2, &inlier_mask);
```

3.3.2 Visualization of Matching Results

To qualitatively evaluate the matching quality, we visualize the inliers and outliers using a dual-pass drawing approach. All initial matches are first rendered in red (representing potential outliers), and then the confirmed inliers are overlaid in green.

The visualization logic utilizes the `cv::DrawMatchesFlags::DRAW_OVER_OUTIMG` flag to perform this overlay, as implemented below:

Listing 4: Visualization of inlier (green) and outlier (red) correspondences

```
std::vector<char> char_inlier_mask{inlier_mask.begin(), inlier_mask.end()};

// Pass 1: Draw all matches in Red (Outliers)
cv::drawMatches(img_1, keypoints_1, img_2, keypoints_2, good_matches,
    img_matches,
cv::Scalar(0, 0, 255), cv::Scalar(0, 0, 255), std::vector<char>(),
DrawMatchesFlags::NOT_DRAW_SINGLE_POINTS);

// Pass 2: Overlay confirmed inliers in Green
cv::drawMatches(img_1, keypoints_1, img_2, keypoints_2, good_matches,
    img_matches,
cv::Scalar(0, 255, 0), cv::Scalar(0, 255, 0), char_inlier_mask,
DrawMatchesFlags::NOT_DRAW_SINGLE_POINTS | DrawMatchesFlags::DRAW_OVER_OUTIMG);

cv::imwrite(inlier_outliner.jpg, img_matches);
```

The resulting Figure 3 demonstrates the efficacy of the rejection algorithm in filtering out spatially inconsistent matches.

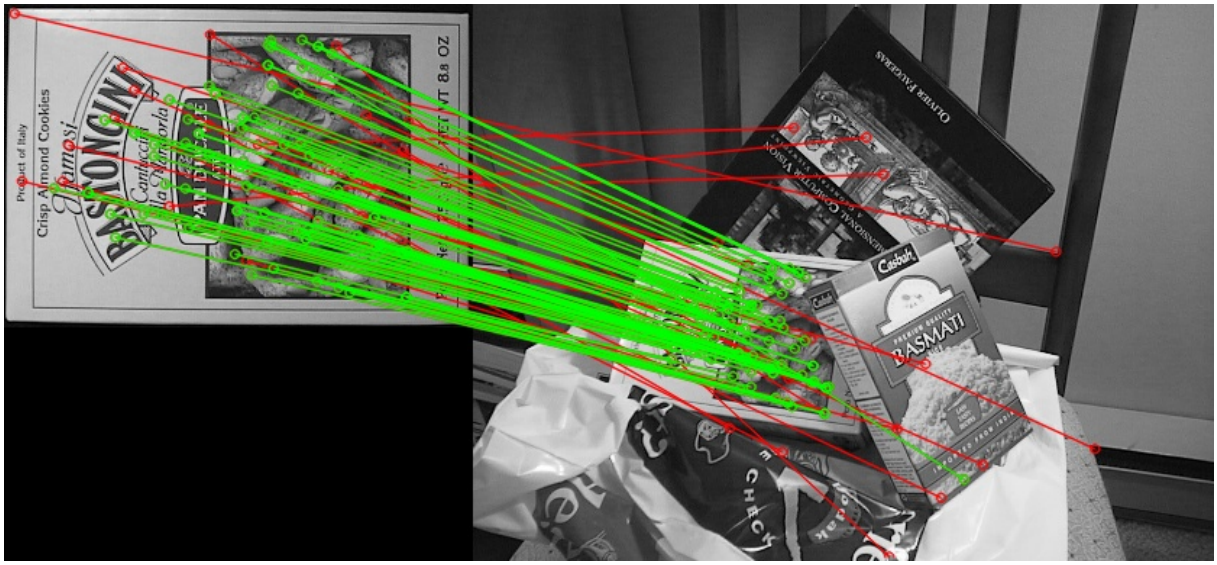


Figure 3: Visualization of SIFT feature matching quality. Green lines indicate inlier correspondences conforming to geometric constraints, while red lines represent rejected outliers.

3.3.3 Statistical Performance Metrics

To quantitatively compare SIFT with other feature tracking algorithms, we compute several average performance metrics over the image sequence. Let be the current number of processed frames. The cumulative average of a metric (such as the inlier ratio) is updated recursively:

$$\bar{M} * N = \frac{\bar{M} * N - 1 \cdot (N-1) + M_N}{N} \tag{16}$$

The *Inlier Ratio* (), a critical measure of the descriptor's discriminative power, is defined as:

$$\eta = \frac{\text{Number of Inliers}}{\text{Number of Good Matches}} \tag{17}$$

These statistics, including the average number of detected keypoints, matches, and inlier ratios, are summarized in Table **??** in the following section.

3.4 Comparative Analysis of Feature Matching Algorithms on Real Data

In this section, we evaluate and compare the performance of several state-of-the-art feature detection and description frameworks: SIFT, SURF, ORB, and a combination of FAST with BRIEF. These algorithms are tested on both discrete image pairs and real-world video datasets to assess their robustness, density, and precision in dynamic environments.

3.4.1 Theoretical Background of Evaluated Frameworks

While SIFT provides a baseline for scale and rotation invariance, other algorithms offer trade-offs between computational complexity and matching quality:

- **SURF (Speeded-Up Robust Features)**: Designed as a high-performance alternative to SIFT, SURF utilizes the determinant of the Hessian matrix for keypoint localization. It accelerates the computation by approximating Gaussian second-order derivatives with **box filters**. Given an integral image , the box filter response can be computed in constant time regardless of size:

$$I_\Sigma(\mathbf{x}) = \sum_{i=0}^{i \le x} \sum_{j=0}^{j \le y} I(i,j) \tag{18}$$

  ‘‘‘

- **ORB (Oriented FAST and Rotated BRIEF)**: A binary descriptor-based framework optimized for real-time SLAM. It enhances the FAST detector by calculating the intensity centroid to provide an orientation θ:

$$\theta = \text{atan2}(m_{01}, m_{10}) \tag{19}$$

where m_{pq} are the geometric moments. Matching is performed using the **Hamming distance**, which is significantly faster than the Euclidean distance used in SIFT/SURF.

- **FAST & BRIEF**: This combination prioritizes raw speed. FAST identifies corners by testing a circular neighborhood of 16 pixels, while BRIEF constructs a binary string via simple intensity comparisons. Note that this basic combination lacks the scale and rotation invariance inherent in ORB.

  “‘

3.4.2 Implementation for Binary Descriptors

For ORB and FAST+BRIEF, the matching logic differs from SIFT as it requires the Hamming norm. The implementation utilizes the `cv::BFMatcher` with the `NORM_HAMMING` metric. Consistent with our previously defined Lowe's ratio test (), the matching logic is implemented as follows:

Listing 5: Brute-Force Matching for Binary Descriptors

```
if (descriptors_1.empty() || descriptors_2.empty()) return;
// Use Hamming distance for binary descriptors
cv::BFMatcher bf_matcher(cv::NORM_HAMMING);
bf_matcher.knnMatch(descriptors_1, descriptors_2, *matches, 2);

for (size_t i = 0; i < matches->size(); i++) {
if ((*matches)[i].size() >= 2) {
const cv::DMatch& n1 = (*matches)[i][0];
const cv::DMatch& n2 = (*matches)[i][1];
// Apply Lowe's Ratio Test
if (n1.distance < 0.8 * n2.distance) {
good_matches->push_back(n1);
}
}
}
```

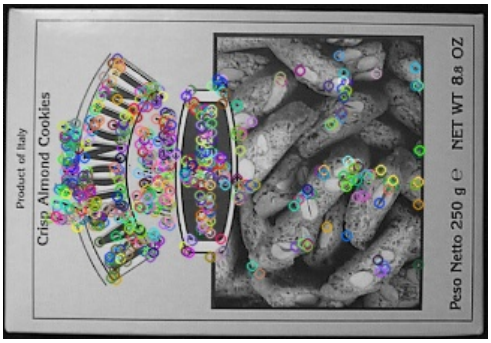
3.4.3 Experimental Results on Static Pairs

Figure 4 illustrates the performance of the ORB algorithm across different scenarios within the dataset.

The first experiment evaluates a single pair of frames to determine the initial discriminative power of each descriptor. The statistical results are summarized in Table 1.

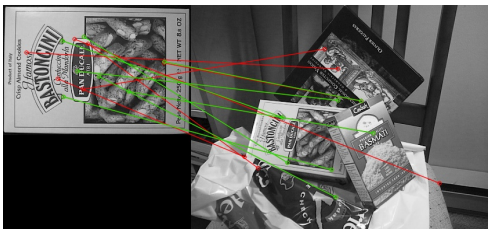
3.4.4 Performance Evaluation on Real Datasets

To test the algorithms under realistic conditions, we utilize two video sequences: a standard 30fps indoor dataset and a smooth trajectory dataset (`vnav-lab5`).

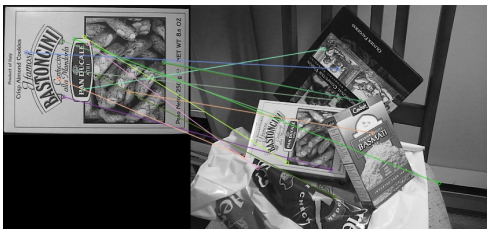


(a) Keypoints in Img 1

(b) Keypoints in Img 2



(c) Good matches



(d) Inliers and outliers

Figure 4: Experimental results of the ORB algorithm

Table 1: Performance comparison on a single image pair.

Statistics	Approach			
	SIFT	SURF	ORB	FAST
# of Keypoints in Img 1	603	997	500	2266
# of Keypoints in Img 2	969	1822	500	3653
# of Matches	603	997	500	2266
# of Good Matches	98	138	13	41
# of Inliers	76	91	7	17
Inlier Ratio	0.7751	0.6594	0.5384	0.4146

Table 2: Statistics for Dataset: $30\text{fps}_4 24x240_2 018 - 10 - 01 - 18 - 35 - 06.bag$

Statistics	Approach for Dataset 30fps_424x240_2018-10-01-18-35-06.bag			
	SIFT	SURF	ORB	FAST
# of Keypoints in Img 1	274	382	318	763
# of Keypoints in Img 2	274	382	318	763
# of Matches	274	382	318	763
# of Good Matches	116	232	232	572
# of Inliers	105	215	228	544
Inlier Ratio	0.9123	0.9271	0.9792	0.9452

3.4.5 Comparative Discussion

Based on the empirical data collected (Table 2 and Table 3), we conclude the following:

1. **Keypoint Density**: The **FAST** detector consistently provides the highest density of keypoints, making it suitable for applications requiring dense environment

Table 3: Statistics for Dataset: vnav-lab5-smooth-trajectory

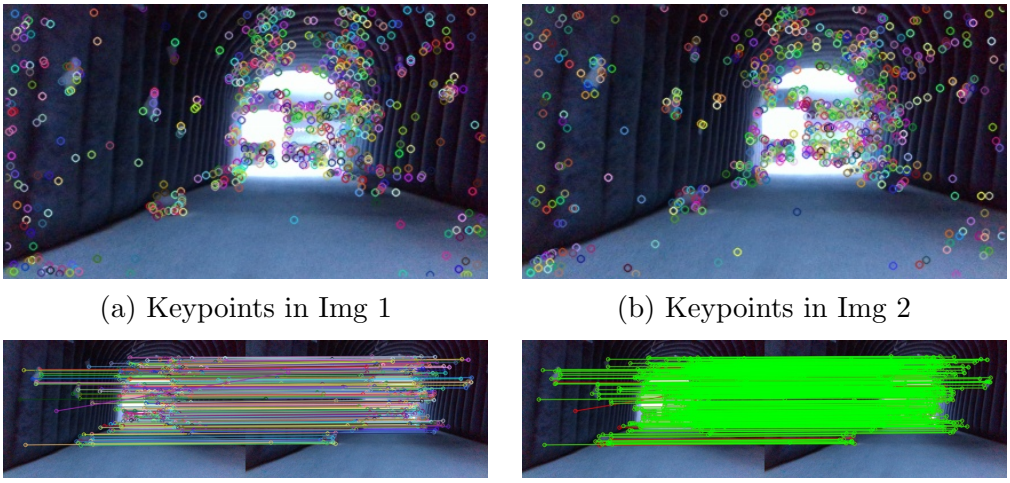
Statistics	Approach for Dataset vnav-lab5-smooth-trajectory			
	SIFT	SURF	ORB	FAST
# of Keypoints in Img 1	138	498	179	501
# of Keypoints in Img 2	137	476	179	499
# of Matches	138	498	179	499
# of Good Matches	54	242	145	297
# of Inliers	37	177	139	258
Inlier Ratio	0.6368	0.8138	0.9567	0.8686

mapping. In contrast, **LK** and **SIFT** yield the sparsest results.

2. **Matching Robustness**: **ORB** exhibits the most stable performance across different datasets. While **LK** maintains the highest inlier ratio (in stable sequences), it tracks significantly fewer points. Notably, ORB's inlier ratio remains high () even in the smooth trajectory dataset where SIFT's precision drops to .

3. **Efficiency Considerations**: Binary descriptors (ORB/FAST) utilize the Hamming distance, offering a significant computational advantage for real-time robotic systems. While SIFT/SURF provide high descriptive power, their reliance on L2-norm matching makes them less viable for resource-constrained platforms.

The Figure 5 illustrates how FAST performe on the dataset 30mps.



(a) Keypoints in Img 1 (b) Keypoints in Img 2

(c) Good matches (d) Inliers and outliers

Figure 5: Experimental results of the FAST algorithm on video dataset

3.5 Feature Tracking: Lucas Kanade Tracker

So far we have worked with descriptor-based matching approaches. As you have seen, these approaches match features by simply comparing their descriptors. Alternatively, feature tracking methods use the fact that, when recording a video sequence, a feature will not move much from frame to frame. We will now use the most well-known differential feature tracker, also known as Lucas-Kanade (LK) Tracker.

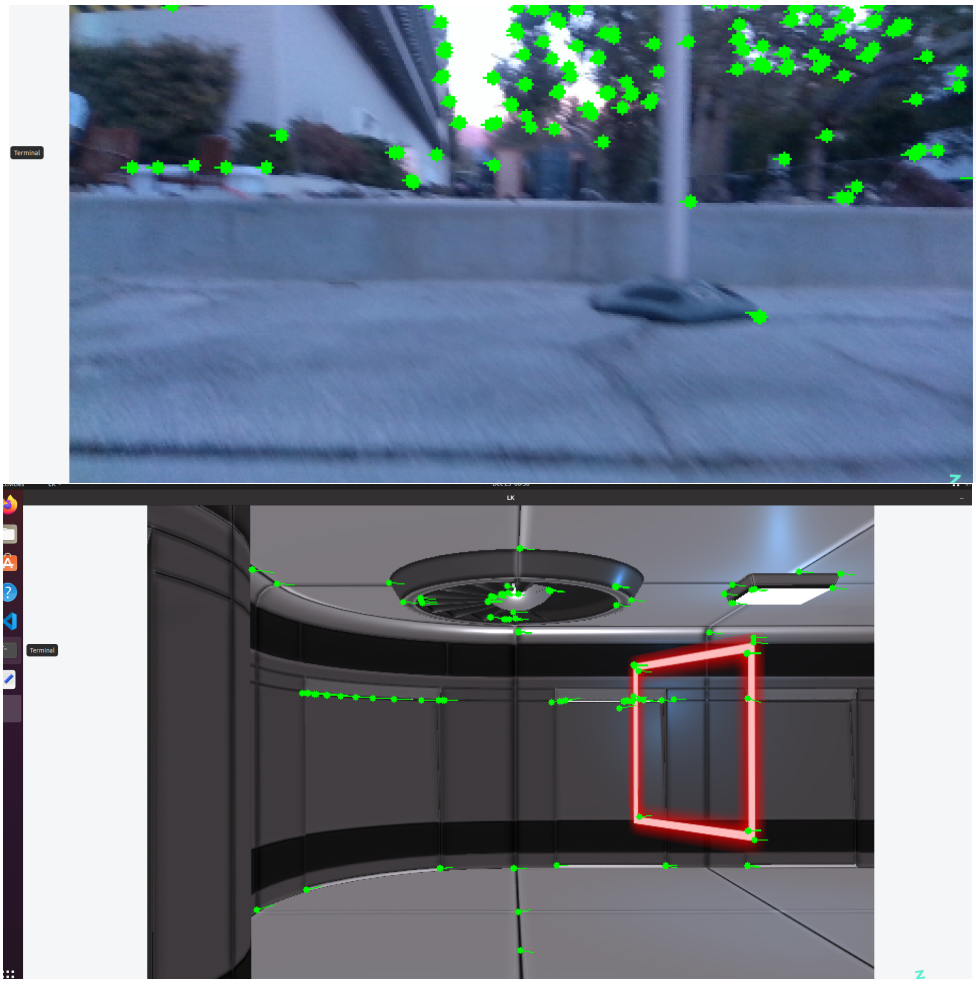


Figure 6: This two graph shows the result of LK algo in two different datasets.

The Optical Flow is the pattern of apparent motion of objects, surfaces, and edges in a visual scene caused by the relative motion between an observer (like a camera) and the scene. In computer vision, it is represented as a 2D vector field where each vector shows the displacement of a pixel from one frame to the next.

Optical flow estimation is primarily built on the Brightness Constancy Assumption: the idea that the intensity of a specific point on an object remains constant between consecutive frames, even as it moves.Mathematically, if $I(x, y, t)$ is the intensity of a pixel at position (x, y) and time t, and it moves by $(\Delta x, \Delta y)$ over time Δt, we assume:

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$$

By applying a Taylor series expansion, we derive the Optical Flow Constraint Equation:

$$f_x u + f_y v + f_t = 0$$

f_x, f_y: Spatial gradients (how brightness changes across the image).f_t: Temporal gradient (how brightness changes over time). u, v: The unknown horizontal and vertical velocities we want to solve for.

Because the equation above has two unknowns (u and v) but only one equation, researchers use different strategies to solve it:Sparse Optical Flow (e.g., Lucas-Kanade): Only tracks a

select set of distinct pixels (like corners). It assumes that all pixels in a small neighborhood move with the same velocity to provide enough equations to solve for u and v.

The LK tracker operates under the assumption of Brightness Constancy, Small Motion, and Spatial Coherence. It assumes that pixel intensities remain constant over time, motion is incremental between frames, and that local pixel neighborhoods move as a single rigid unit. These assumptions allow the algorithm to overdetermine the optical flow equations and solve for motion vectors using a least-squares approach.

We implement this as here

Listing 6: The whole of LK features

```
cv::Mat gray;
   cv::cvtColor(frame, gray, cv::COLOR_BGR2GRAY);

   // 2.
   if (prev_frame_.empty()) {
       cv::goodFeaturesToTrack(gray, prev_corners_, 200, 0.01, 10, cv::Mat(),
  3, true, 0.04);
       gray.copyTo(prev_frame_);
       return;
   }

   // 3. LK
   std::vector<cv::Point2f> curr_corners;
   std::vector<uchar> status;
   std::vector<float> err;
   cv::calcOpticalFlowPyrLK(prev_frame_, gray, prev_corners_, curr_corners,
  status, err);

   // 4.  ( Matches)
   std::vector<cv::Point2f> matched_prev, matched_curr;
   for (size_t i = 0; i < status.size(); i++) {
       if (status[i]) {
           matched_prev.push_back(prev_corners_[i]);
           matched_curr.push_back(curr_corners[i]);
       }
   }

   // 5.  ( Inliers)
   unsigned int num_inliers = 0;
   if (matched_curr.size() >= 8) {
       std::vector<uchar> inlier_mask;
       inlierMaskComputation(matched_prev, matched_curr, &inlier_mask);
       for (uchar v : inlier_mask) if (v) num_inliers++;
   }

   // 6.
   num_samples_++; //
   float const n = static_cast<float>(num_samples_);

   //  ( Avg )
```

```
avg_num_keypoints_img1_ += (static_cast<float>(prev_corners_.size()) -
avg_num_keypoints_img1_) / n;
avg_num_matches_ += (static_cast<float>(matched_curr.size()) -
avg_num_matches_) / n;
avg_num_inliers_ += (static_cast<float>(num_inliers) - avg_num_inliers_) /
n;

if (!matched_curr.empty()) {
    float current_ratio = static_cast<float>(num_inliers) / matched_curr.
size();
    avg_inlier_ratio_ += (current_ratio - avg_inlier_ratio_) / n;
}

// 7.  ()
std::cout << \rLK␣Avg␣Stats:␣[Keypoints:␣ << (int)avg_num_keypoints_img1_
          << ]␣[Matches:␣ << (int)avg_num_matches_
          << ]␣[Inliers:␣ << (int)avg_num_inliers_
          << ]␣[Ratio:␣ << std::fixed << std::setprecision(2) <<
avg_inlier_ratio_ * 100 << %] << std::flush;

// 8.
show(frame, matched_prev, matched_curr);
gray.copyTo(prev_frame_);
prev_corners_ = matched_curr;

//  (Replenish)
if (prev_corners_.size() < 100) {
    std::vector<cv::Point2f> new_corners;
    cv::goodFeaturesToTrack(gray, new_corners, 200 - prev_corners_.size(),
0.01, 10, cv::Mat(), 3, true, 0.04);
    prev_corners_.insert(prev_corners_.end(), new_corners.begin(),
new_corners.end());
}
```

1. Initialization and Feature Detection The tracker first converts the input frame to grayscale. If it is the very first frame, it uses cv::goodFeaturesToTrack (configured here to use the Harris Corner Detector) to identify prominent points to track. This ensures the tracker starts with "strong" features that are easier to follow under the spatial coherence assumption.

2. Lucas-Kanade Tracking The core logic resides in cv::calcOpticalFlowPyrLK. This function implements the Iterative Lucas-Kanade method with Pyramids. Pyramids: By building a Gaussian pyramid, the tracker can handle larger pixel displacements that would otherwise violate the "small motion" assumption. Status Vector: The status output is crucial; it marks which points were successfully found in the new frame and which were lost due to occlusion or lighting changes.

3. Geometric Verification (Inlier Computation) Once matches are found, the code performs an Inlier Mask Computation (typically using RANSAC and the Fundamental/Essential matrix). This filters out "bad matches"—points that might have been tracked successfully by the LK algorithm but do not move consistently with the overall scene

geometry (e.g., a moving person in the background or a tracking error).

4. Recursive Statistical Gathering Because this tracker operates on a video stream rather than a static pair of images, it uses a running average to calculate statistics:

$$\text{Avg}_{new} = \text{Avg}_{old} + \frac{\text{Current} - \text{Avg}_{old}}{n}$$

This provides a stable representation of performance (Keypoints, Matches, and Inlier Ratios) over the entire duration of the dataset, which is what you used for your comparison table.

5. Feature Replenishment One common issue with LK trackers is "feature drop-out," where points eventually move out of frame or are lost. The final block of code checks if the number of tracked features has fallen below a threshold (100 points). If so, it re-runs the Harris detector to replenish the feature set, ensuring the tracker remains robust throughout the video sequence.

The Table 4 and Table 5 add a new column of LK, show how the LK performs on two different dataset.

Table 4: Statistics for Dataset: $30\text{fps}_4 24x240_2 018 - 10 - 01 - 18 - 35 - 06.bag$

Statistics	Approach for Dataset 30fps_424x240_2018-10-01-18-35-06.bag				
	SIFT	SURF	ORB	FAST	LK
# of Keypoints in Img 1	274	382	318	763	127
# of Keypoints in Img 2	274	382	318	763	127
# of Matches	274	382	318	763	126
# of Good Matches	116	232	232	572	126
# of Inliers	105	215	228	544	125
Inlier Ratio	0.9123	0.9271	0.9792	0.9452	0.9982

Table 5: Statistics for Dataset: vnav-lab5-smooth-trajectory

Statistics	Approach for Dataset vnav-lab5-smooth-trajectory				
	SIFT	SURF	ORB	FAST	LK
# of Keypoints in Img 1	138	498	179	501	125
# of Keypoints in Img 2	137	476	179	499	125
# of Matches	138	498	179	499	125
# of Good Matches	54	242	145	297	121
# of Inliers	37	177	139	258	116
Inlier Ratio	0.6368	0.8138	0.9567	0.8686	0.9575

Based on the statistics presented in Table 4 and Table 5, several key distinctions can be observed between the Lucas-Kanade (LK) optical flow approach and feature-based descriptors (SIFT, SURF, ORB, and FAST):

- **Superior Precision and Inlier Ratio:** The most significant observation is that the **LK approach consistently achieves the highest Inlier Ratio** across both datasets (0.9982 in Table 4 and 0.9575 in Table 5). Unlike SIFT or SURF, which search for global matches and often produce incorrect pairings, LK relies on local differential tracking, which inherently filters out "wild" mismatches.

- **Quantity vs. Quality:** Feature detectors like **FAST and SURF** generate a high volume of keypoints (up to 763), but a large percentage are discarded during the filtering process. In contrast, **LK tracks a smaller, more stable set** of points (around 125-127). Although LK provides lower density, the "survival rate" from initial keypoints to inliers is nearly 100%, demonstrating exceptional reliability per point.

- **Robustness to Trajectory Types:** In the "smooth-trajectory" dataset (Table 5), the performance of SIFT drops significantly (Inlier Ratio of 0.6368). However, **LK maintains a very high ratio** (0.9575). This suggests that for sequential video data where frame-to-frame motion is incremental, LK's assumption of local brightness constancy is highly effective compared to descriptor-based matching.

In summary, while feature-based methods are better suited for large-scale displacement or re-localization, the **LK tracker** is the most precise and efficient method for continuous, smooth motion tracking as evidenced by the near-perfect inlier statistics.

4 Dense Optical Flow Estimation

4.1 Theoretical Background

While sparse optical flow methods like Lucas-Kanade (LK) tracker estimate motion for a specific set of feature points, dense optical flow techniques aim to compute the displacement vector for every pixel in the frame. In this experiment, we implement the **Farneback Algorithm**, which is based on polynomial expansion.

The core principle of Farneback's method is to approximate the neighborhood of each pixel with a quadratic polynomial:

$$f(\mathbf{x}) \approx \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x} + c$$

where $\mathbf{x}$ is the local coordinate, $\mathbf{A}$ is a symmetric matrix, $\mathbf{b}$ is a vector, and c is a scalar. By observing how this polynomial coefficients shift between consecutive frames f_1 and f_2, the displacement $\mathbf{d}$ can be estimated.

Given a new signal f_2 constructed by shifting f_1 by $\mathbf{d}$:

$$f_2(\mathbf{x}) = f_1(\mathbf{x} - \mathbf{d})$$

Equating the coefficients of the quadratic forms leads to the following relationship:

$$\mathbf{A}_2 = \mathbf{A}_1$$

$$\mathbf{b}_2 = \mathbf{b}_1 - 2\mathbf{A}_1\mathbf{d}$$

From which the displacement $\mathbf{d}$ can be solved via the linear system:

$$2\mathbf{A}_1\mathbf{d} = -(\mathbf{b}_2 - \mathbf{b}_1)$$

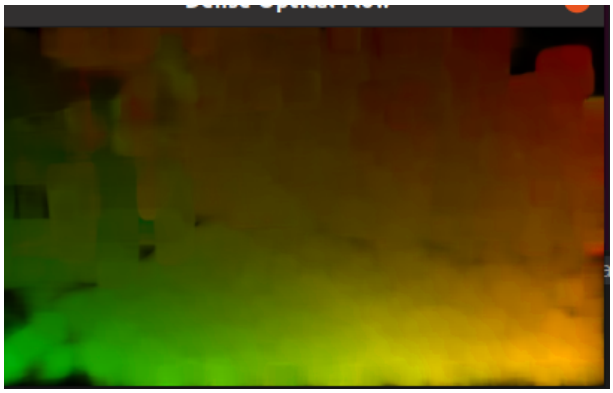


Figure 7: The direction of motion is encoded by the Hue (color), while the velocity magnitude is represented by the Value (brightness) of the pixel.

4.2 Implementation Details

The implementation is integrated into a ROS node (`self_flow.cpp`). The process follows a structured pipeline as illustrated in our implementation:

1. **Preprocessing:** Incoming ROS image messages are converted to OpenCV `Mat` format using `cv_bridge::toCvCopy`. The frames are converted to grayscale to satisfy the input requirements of the algorithm.

2. **Flow Computation:** The function `cv::calcOpticalFlowFarneback` is employed. We utilize a multi-scale pyramid approach to handle large displacements:

   - `pyr_scale`: 0.5 (Classical image pyramid).
   - `levels`: 3 (Number of pyramid layers).
   - `winsize`: 15 (Averaging window size).

3. **Visualization (HSV Mapping):** To intuitively represent the 2D flow vectors, we map the flow from Cartesian coordinates (u, v) to Polar coordinates (magnitude, angle).

As shown in Figure 7, the direction of motion is encoded by the **Hue** (color), while the velocity magnitude is represented by the **Value** (brightness) of the pixel.

4.3 Experimental Results and Analysis

The dense flow field provides a much richer motion description compared to sparse trackers.

As shown in Figure 8, the algorithm successfully captures the global motion of the scene. Unlike the LK tracker, which might fail in textureless regions, the Farneback method maintains a continuous flow field by incorporating neighborhood polynomial expansion. However, this comes at a higher computational cost per frame.

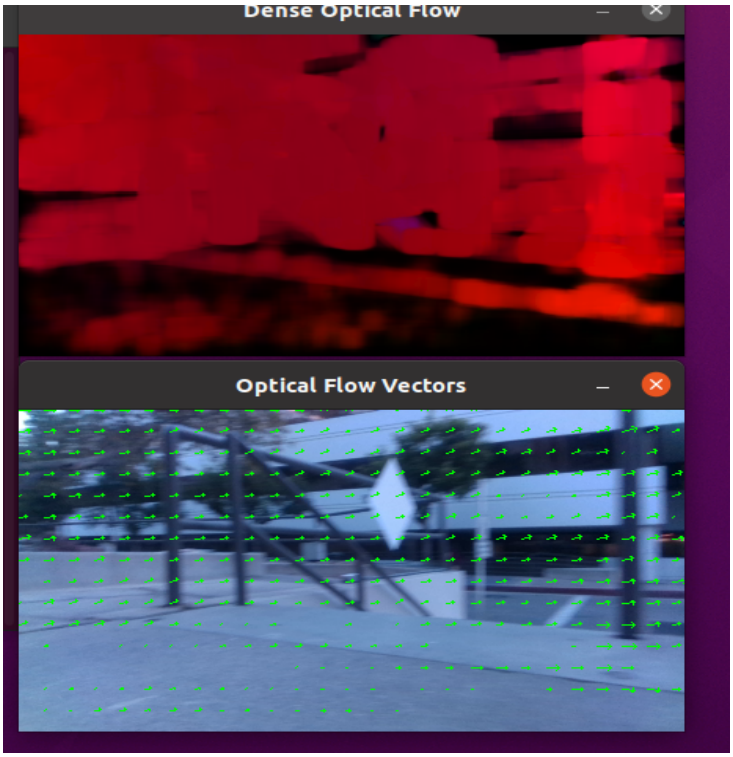


Figure 8: Visualization of Dense Optical Flow using Farneback Algorithm. The colors indicate the direction of movement for each pixel.

4.4 Discussion on Robustness

To ensure the stability of the ROS node, we implemented a memory-safe approach by using `toCvCopy` instead of `toCvShare`. Additionally, explicit type conversion was applied during HSV merging:

$$\text{Hue} = \frac{\text{Angle}}{2}, \quad \text{Value} = \text{Normalize}(\text{Magnitude})$$

This ensures that the output remains within the `CV_8UC1` range, preventing runtime overflows or memory access violations common in dense flow visualizations.

5 Discussion

This lab is an introduction to the Computer vision. In this lab we introduce 4 kinds of detect and match algorithm and give out the basic comparision and conclusion to them.

While there was no active hands-on implementation of these algorithms, it was certainly interesting to read about their principles. Previously, I have been exposed to more traditional and older computer knowledge, such as the internals of systems and data structures. Pictures are also expressed numerically in computers and are two-dimensional, so how to deal with them and further compute them is an interesting topic. But my surprise is still more in the power of modern computers to process such complex images and even streams of images, while previously it was difficult to understand why it was done when yolov neural networks were called directly to do object detection, and when I heard some-

thing about convolutional and deep learning. But this experiment learned unexpectedly suitable for understanding, and there is nothing difficult to explain.

6 Conclusions

This experiment we briefly use the CV algo(SIFT, SURF, ORB, FAST, LK) and record their performance on different dataset. This is some methods how to extract the information from the simple image.